so i have a local repo on my mac that I did a git clone of a repo say X on github (someone else's). Now I have my own repo also on github, call it R. I want to create a folder inside R (say Y) where I want to now upload my version of X. Second, on my "local" X I want to be able to do both : push my changes into R and at the same time be able to pull the source from the actual X on github. Got it? Give me steps to do all of these things clearly.

To do this properly in Git, you need to set up **two remotes** on your local copy of repository X: one pointing to the original repo (to pull updates) and one pointing to folder Y inside your repo R on GitHub (to push your changes). [1]
Here are the clear steps to set this up from your Mac's Terminal.

Step 1: Set up the upstream remote to pull from the original X

Open your Mac **Terminal** and navigate into your existing local clone of X:

bash

```
cd /path/to/your/local/X
```

Use code with caution.

Rename the default remote (which currently points to the original owner's repo X) to upstream: [1]

bash

```
git remote rename origin upstream
```

Use code with caution.

Now, whenever you want to pull the latest changes from the original creator of X, run: [1]

bash

```
git pull upstream main
```

Use code with caution.

(Note: Replace main with master if the original repo uses master as its default branch).

Step 2: Connect your local X to your new repo R (inside folder Y)

You have two main ways to handle putting repository X inside folder Y of your repository R.

Option A: Using a Subtree (Recommended — keeps history and lets you push/pull cleanly)

If you want Git to track folder Y inside your repository R without breaking the inner git structure:

1. Clone or navigate to your local copy of your own repo R:bash

   cd /path/to/your/local/R
2.

   Use code with caution.

3. Create folder Y and place your files there, or use git subtree to link your local X code into folder Y:bash

   git subtree add --prefix=Y /path/to/your/local/X main --squash
4.

   Use code with caution.

5. Push R to GitHub:bash

   git push origin main
6.

   Use code with caution.

Option B: Add R as a second remote (origin) on your local X
If you prefer to push your local X directly to your GitHub repo R (mapping it to folder Y via a dedicated branch or direct push):

1. Add your GitHub repository R as origin in your local X folder:bash

   cd /path/to/your/local/X
2. git remote add origin https://github.com
3.

   Use code with caution.

4. Because X has its own root and R has folder Y, pushing directly can mix file paths unless you isolate them into a branch or use a subdirectory subtree. The standard Git way to push a subdirectory X into a subfolder Y of another repo is using git subtree push:bash

   git subtree push --prefix=. origin r-branch-name
5.

   Use code with caution.

Summary of Daily Commands

- **To pull updates from the original owner (X):**bash

git pull upstream main

-

  Use code with caution.

- **To push your modifications to your repo R (in folder Y):**bash

  git subtree push --prefix=Y origin main

-

  Use code with caution.

  (Or standard git push origin <branch> if you structured R with X living inside folder Y).

If you'd like, tell me:

- Do you want to **merge/blend** the history of X into R, or just drop the files into folder Y?
- Are you using **HTTPS or SSH** for your GitHub authentication?

I can give you the exact copy-paste commands for your specific setup.

AI responses may include mistakes. Learn more

- 
-

  Stack Overflow

Cloning a repo from someone else's Github and pushing it to a ...
Forking Workflow: A recommended method involves creating a new repository on GitHub (without initializing it), cloning the origina...

No I don't really want to merge/blend the upstream changes of X into R, just whatever changes that I do to my X should go to R. Does that change anything?

Yes, that simplifies things. Since you only want to push **your personal version of**

X into folder Y inside your repository R (and never pull the original author's updates directly into R), you should use **Git Submodules**.
This keeps the two repositories completely independent. Your repo R will simply point to your own modified version of X inside folder Y.
Here are the updated steps to set this up:

Step 1: Create a brand new repository for your version of X on GitHub

Because you want your changes to go to GitHub, you need a place for your personal X code to live on your GitHub account.

1. Go to GitHub and create a new **public or private repository** named my-version-of-X.
2. Leave it completely empty (do **not** initialize it with a README or .gitignore).

Step 2: Configure your local X folder with two remotes

Open your Mac Terminal, navigate to your local X folder, and configure it to talk to both the original author and your new GitHub repo.

bash

```
cd /path/to/your/local/X

# 1. Rename the original creator's remote to 'upstream'
git remote rename origin upstream

# 2. Add your brand new GitHub repo as 'origin'
git remote add origin https://github.com

# 3. Push your current local code to your new GitHub repo
git push –u origin main
```

Use code with caution.

- **To pull future updates from the original author:** git pull upstream main (this updates your local files only).
- **To push your changes to your own account:** git push origin main.

Step 3: Link your version of X inside repository R as folder Y

Now, navigate to your local repository R and inject your version of X exactly into folder Y.

bash

```
cd /path/to/your/local/R

# Add your version of X as a submodule inside a new folder
named Y
git submodule add https://github.com Y

# Commit and push this link to your main repository R
git add .
git commit –m "Added my version of X into folder Y as a
submodule"
git push origin main
```

Use code with caution.

How you will work with this daily

- **To make changes:** Work inside your local X folder normally. When you run git push origin main, your changes go straight to your my-version-of-X repo.
- **To update repository R:** Whenever you push new changes to X, go over to your R folder and run git commit -am "update submodule Y" && git push origin main so R knows to point to your latest commit.

To make sure this matches your exact workflow, tell me:

- Do you prefer to **edit the code of X directly inside folder Y** of your repo R, or do you want to keep editing it in its **original standalone folder** on your Mac?

I can give you the exact terminal commands for whichever workflow you prefer.

AI responses may include mistakes. Learn more

another weird thing .. (I am on a macbook pro). Anywhere on my mac when I do git push origin main I NEVER get asked my username and password (and I have a key associated with this mac tied up to my github etc) but I create this NEW repo in my github (empty) then I git clone it using SSH, then I add a README.MD, commit the change (all good so far) but when I do git push -u origin main, it asks me for my username and password now.. WHY ?

The reason this is happening is that **your terminal is accidentally trying to use HTTPS instead of SSH** for this specific new repository, even though you intended to use SSH.
When you see GitHub asking for a username and password in the terminal, it is a guaranteed sign that the remote URL starts with https:// instead of git@github.com:. Since GitHub discontinued password authentication over HTTPS in 2021, this prompt will actually reject your normal password anyway.
Here is exactly why this happened and how to fix it in 5 seconds.

Why did this happen?

There are two common reasons this sneaky switch happens:

1. **The Copy-Paste Slip:** On the empty GitHub repository page, the "Quick setup" page defaults to the **HTTPS** tab highlighted unless you explicitly click the **SSH** button before copying the clone/remote URL.
2. **Global Git Instead-Of Rules:** You might have a global setting on your Mac that automatically rewrites URLs, or you copied the HTTPS git remote add command from my previous response without swapping it to your SSH string.

How to Fix It Immediately

You do not need to re-clone anything. You just need to tell your local repository to use your SSH key.

1. Open your terminal and navigate to that new repository folder.
2. Run this command to check your current hidden URL:bash

   git remote -v
3.

   Use code with caution.

   You will likely see it starts with https://github.com...
4. Run this command to **force change the remote to use SSH**:bash

   git remote set-url origin git@github.com:YOUR-USERNAME/YOUR-REPO-NAME.git
5.

   Use code with caution.

   (Make sure to replace YOUR-USERNAME and YOUR-REPO-NAME with your actual GitHub username and the new repository's name).
6. Try pushing again:bash

   git push -u origin main
7.

   Use code with caution.

It will now instantly use the SSH key safely tucked away in your MacBook Pro's Keychain, and it won't prompt you for a username or password ever again.

Did switching the remote URL to the git@github.com: format successfully bypass the login prompt? If it still asks for a passphrase, let me know if your SSH key is protected by a password or if it's a standard passphraseless key.

AI responses may include mistakes. Learn more

yeah that worked. Back to the original context. So I have changed my mind. Lets say there is a github repo called lerobot (someone else's) that I have cloned locally. I added some changes to it. Now, I create a repo called r-playground on my github, ok? Empty for now. I cloned it, added a README.md etc. Now, I "copied" my local lerobot inside my local r-playground. Now I want to do two things : make my r-playground on my gihthub always contain my version of lerobot, but also want the ability to keep updating my lerobot with the actual github lerobot so that I am always on the latest code changes. But currently when I cp -r'd the local lerobot also contains the .git folder which makes it point to the github lerobot. But I want THAT to be upstream and the origin to be my github's r-playground. BUT I also want to "remember" the notion of what upstream this local lerobot folder of r-playground should automatically point to, if I say (once I uploaded everything) did a git clone of the final r-playground somewhere else. How do I do all this? step by step please.

To achieve exactly what you want—where your inner lerobot folder remembers its original upstream source even if you clone r-playground onto a brand-new computer—you **must use a Git Submodule**.
If you just copy-paste the files, Git treats them as static files and completely

forgets where they came from. A submodule tells Git: *"This folder is actually an independent repository, and here is its original URL."* [4]
Because you already copied the folder over, we need to clean that up first so Git doesn't get confused. Here are the step-by-step commands to set this up cleanly from your Mac.

Step 1: Clean up the copied folder

Before turning it into a proper submodule, we need to make sure your main repository isn't tracking the raw .git folder you copied.

1. Open your Terminal and navigate to your **local r-playground** folder:bash

   cd /path/to/your/local/r-playground
2.

   Use code with caution.

3. Completely remove the lerobot folder you manually copied so we can start fresh:bash

   rm -rf lerobot
4.

   Use code with caution.

Step 2: Add lerobot as a proper Submodule using SSH

Instead of copying files, you will tell your r-playground repository to link directly to the official lerobot source repository using its GitHub URL.

1. Run this command inside your r-playground folder:bash

   git submodule add https://github.com lerobot
2.

   Use code with caution.

   (Note: This creates a fresh lerobot folder and downloads the code. It also creates a hidden file named .gitmodules which is the magic file that "remembers" where this folder came from forever [4]).
3. Commit and push this setup to your GitHub r-playground:bash

   git add .
4. git commit -m "Add lerobot as an official submodule"
5. git push origin main
6.

   Use code with caution.

Step 3: Configure lerobot to point to your r-playground for your changes

Right now, the inner lerobot folder is pointing to the original author's repository as its origin. Because you don't have permission to push changes directly to the official Hugging Face repository, you need to tell it to push to your r-playground instead.

1. Navigate **inside** the newly created inner lerobot folder:bash

   cd lerobot

2.

   Use code with caution.

3. Change its origin to point to your r-playground repo on GitHub using **SSH** so it never prompts you for a password:bash

   git remote set-url origin git@github.com:YOUR-USERNAME/r-playground.git
4.

   Use code with caution.

5. Rename the original repository remote to upstream so you can pull future updates from it:bash

   git remote rename origin upstream
6.

   Use code with caution.

   (If it throws an error saying upstream already exists, you can skip this step).

How to use this daily

1. To pull the latest official updates from the original lerobot:
Whenever you want to pull the latest changes from the official authors into your playground:

bash

```
cd /path/to/your/local/r-playground/lerobot
git pull upstream main
```

Use code with caution.

2. To push your own changes:
When you write code inside lerobot and want to save it to your own GitHub:

1. First, commit and push inside the lerobot folder:bash

   cd /path/to/your/local/r-playground/lerobot
2. git add .
3. git commit -m "My custom changes to lerobot"
4. git push origin main
5.

   Use code with caution.

6. Second, go back to your main folder and tell r-playground to save the new pointer:bash

   cd /path/to/your/local/r-playground
7. git add lerobot
8. git commit -m "Update lerobot pointer"
9. git push origin main
10.

    Use code with caution.

What happens if you clone r-playground somewhere else?

Because we used a submodule, if you ever go to a different computer and run a standard git clone, the lerobot folder will initially look empty.
To bring it to life on a new machine, you just run this single command inside your new clone:

bash

```
git submodule update --init --recursive
```

Use code with caution.

Git will read the hidden .gitmodules file, remember exactly where the official lerobot repository is, and download the exact version you were working on automatically!

Did you already write a lot of custom code inside that copied lerobot folder that you need to save before running rm -rf lerobot? If so, tell me, and I can give you a quick command to backup your files first so you don't lose your work.

AI responses may include mistakes. Learn more

I dont think you understand what I meant. I want to carry MY changes of lerobot where I will be experimenting a lot. If I add it as a submodule that points to the original lerobot always, then how will it keep a track of my changes which are bound to grow?

You are completely right, and I see where the confusion happened.
If you use a standard submodule pointing directly to the original lerobot, any local changes you make will eventually get wiped out, overwritten, or won't be saved anywhere on GitHub. That is absolutely **not** what you want for an active experimentation workspace.
Because you are working with **lerobot**—and will be heavily customizing code, configuration files, and weights—you need your folder Y inside repository R to point to **your own personal fork** of lerobot.
Here is exactly how to structure this so your experiments are safely tracked in your GitHub account, while still letting you pull new updates from the official lerobot repository.

The Correct Architecture

Instead of pointing folder Y to the original lerobot, you will point it to a **personal copy** on your GitHub.

text

```
[Official lerobot] (Upstream)
       │
       ▼ (Pull official updates into your local machine)
[Your Mac: Folder Y]
       │
       ▼ (Push your custom experiments)
[Your GitHub: repo-R / folder-Y] ──► points to ──► [Your GitHub: my-custom-lerobot]
```

Use code with caution.

Step-by-Step Setup

1. Fork lerobot on GitHub

1. Go to the official huggingface/lerobot repository on GitHub.
2. Click the **Fork** button in the top-right corner to create a copy under your own GitHub account (e.g., ://github.com).

2. Clear out your old local experiment folder (If needed)

If you already have a messy local clone of lerobot that isn't connected properly, it is easiest to start fresh. Move your modified files to a temporary desktop folder, delete the old folder, and navigate to your main repository R in your Mac Terminal:

bash

```
cd /path/to/your/local/R
```

Use code with caution.

3. Add YOUR fork as the submodule (Folder Y)

Run this command using your **SSH URL** (since your Mac is configured for SSH) to inject your fork into folder Y:

bash

```
git submodule add git@github.com:YOUR-USERNAME/lerobot.git Y
```

Use code with caution.

(Replace YOUR-USERNAME with your GitHub username).

4. Configure Folder Y to pull from the official lerobot
Now, go *inside* that new folder Y so we can tell it where the original creator's code lives:

bash

```
cd Y

# Add the official HuggingFace repo as a tracking remote named 'upstream'
git remote add upstream git@github.com:huggingface/lerobot.git
```

Use code with caution.

How You Save and Update Your Experiments (Daily Workflow)

Now that everything is linked up cleanly, here is how you manage your code daily without breaking anything.

Scenario A: You wrote a lot of custom experiment code in Y and want to save it
Since folder Y is its own Git repository under the hood, you commit and push your changes directly from inside it:

bash

```
cd /path/to/your/local/R/Y

git add .
git commit –m "Added my custom training scripts and network modifications"
git push origin main
```

Use code with caution.

*This pushes your code directly to **your GitHub fork**, keeping a perfect history of your growing experiments.*

Scenario B: HuggingFace releases a massive update to lerobot that you want
If the official team fixes a bug or adds a cool new feature you need, you can pull it directly into your workspace without losing your files:

bash

```
cd /path/to/your/local/R/Y

# Pull the fresh updates from the official source
git pull upstream main
```

Use code with caution.

Git will automatically merge their updates into your files. If there are conflicts in files you changed, Git will ask you to fix them just like a normal team workflow.

Scenario C: Committing the link in your Main Repo R

Whenever you update your submodule code, your main repository R will notice that folder Y is pointing to a newer commit. To save this state on your main project:

bash

```
cd /path/to/your/local/R

git add Y
git commit -m "Updated lerobot experiment folder to latest commit"
git push origin main
```

Use code with caution.

AI responses may include mistakes. Learn more

there is still something you don't understand. r-playground will host lerobot and a couple of other open source repos and I will be interconnecting them and doing various things. (That is why I have a r-playground and that is why lerobot was a subfolder). So how do I make my lerobot a subfolder of r-playground (knowing there will be more "forked peers") and actually do this. Think through this carefully and generate the right list of commands and steps / sequences.

I see exactly what you are building now. You are creating a **monorepo-style**

workspace (r-playground) that acts as an umbrella container holding multiple independent open-source repositories (lerobot, and others later) as subfolders. You want to modify them, connect them, and have your main r-playground repo on GitHub track *everything*—including your custom code changes inside those subfolders—while still letting those subfolders pull updates from their original upstream authors.

To do this perfectly, you **cannot** point a subfolder's origin to r-playground. In Git, a repository cannot push its own internal code directly into a subdirectory of itself without corrupting the Git tree.

Instead, the cleanest, industry-standard solution for a playground of interconnected open-source repos is to use **Git Trees (Subtrees)** or a **Single Monorepo with Multiple Remotes**. Since you explicitly want this setup to **"remember" the upstream URLs automatically** when cloned onto a new machine, **Submodules** are still the correct mechanism, but they must be structured via your own GitHub forks.

Here is the exact step-by-step blueprint and sequence of commands to set this up cleanly on your Mac.

Phase 1: Fork lerobot on GitHub First

Because you are changing code inside lerobot, those changes need a destination on your GitHub account.

1. Go to GitHub and navigate to the official huggingface/lerobot repository.
2. Click the **Fork** button in the top right to create YOUR-USERNAME/lerobot on your profile.
3. *(Repeat this fork step in the future for any other peer repos you bring into your playground)*.

Phase 2: Clean Your Local Workspace

Let's clear out the manually copied folder so Git doesn't run into conflicts.

1. Open Terminal and navigate to your local r-playground:bash

   cd /path/to/your/local/r-playground
2.

   Use code with caution.

3. Remove the manually copied lerobot folder:bash

   rm -rf lerobot
4.

   Use code with caution.

Phase 3: Add lerobot as a Subfolder Tracking Your Fork

Now, we inject lerobot as a subfolder inside r-playground. We will point it to **your fork** so it can save your changes, but it will keep a record of where it came from.

1. Run this command inside r-playground (use SSH to avoid password prompts):bash

   git submodule add git@github.com:YOUR-USERNAME/lerobot.git lerobot
2.

   Use code with caution.

   This downloads your fork into a subfolder named lerobot and creates a .gitmodules file that permanently records this relationship.
3. Commit and push this structure to your main r-playground on GitHub:bash

   git add .
4. git commit -m "Structure: Added lerobot to the playground workspace"
5. git push origin main

6.

Use code with caution.

Phase 4: Tell the lerobot Subfolder About the Original Author

Right now, your lerobot subfolder knows about your fork (origin), but it doesn't know about the original Hugging Face source yet. Let's add it as upstream.

1. Move **inside** the lerobot subfolder:bash

   cd lerobot
2.

   Use code with caution.

3. Add the official repository as the upstream remote:bash

   git remote add upstream https://github.com
4.

   Use code with caution.

The Daily Workspace Workflow

Now your workspace is fully established. Here is how you manage it daily without getting tangled up.

Scenario A: You modified code inside lerobot and want to save everything

When you edit files inside the lerobot subfolder, you need to push the code to your fork, and then update the pointer in your main playground.

1. Save and push your changes inside the lerobot folder:bash

   cd /path/to/your/local/r-playground/lerobot
2. git add .
3. git commit -m "Custom tweaks to lerobot tracking logic"
4. git push origin main
5.

   Use code with caution.

6. Move back up to your main playground and commit the new state of the workspace:bash

   cd /path/to/your/local/r-playground
7. git add lerobot
8. git commit -m "Workspace sync: updated lerobot pointer"
9. git push origin main
10.

    Use code with caution.

Scenario B: You want to pull the latest official updates from Hugging Face

If the original creators of lerobot release an update you want to use:

1. Go into the subfolder and pull from their master branch:bash

   cd /path/to/your/local/r-playground/lerobot
2. git pull upstream main
3.

   Use code with caution.

4. Push those updates to your personal fork so your cloud copy stays updated:bash

   git push origin main
5.

   Use code with caution.

Cloning This Playground to a New Mac in the Future

Because of the .gitmodules file created in Phase 3, your r-playground repository will forever remember exactly how this workspace is tied together. If you clone r-playground somewhere else down the road:

1. Clone your main playground:bash

   git clone git@github.com:YOUR-USERNAME/r-playground.git
2. cd r-playground
3.

   Use code with caution.

4. Run this single command to have Git automatically look at your .gitmodules configuration, find your lerobot fork, create the subfolder, and download the exact version of the code you left off with:bash

   git submodule update --init --recursive
5.

   Use code with caution.

Does this architecture map correctly to how you envision adding your next "forked peer" repositories down the line? If you'd like, I can show you how to write a quick shell script alias for your Mac so you can push or update all peer subfolders simultaneously with a single terminal command.